

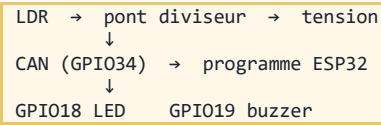
1. SYSTÈME MICROPROGRAMMÉ

Un microcontrôleur exécute un programme stocké en mémoire. Il lit des capteurs et commande des actionneurs.

- Processeur : exécute les instructions, une après l’autre
- Mémoire flash : conserve le programme, même hors tension
- Mémoire RAM : contient les variables pendant l’exécution
- Périphériques : entrées-sorties, CAN, liaison série
- Horloge : cadence l’exécution

Au démarrage : setup() une seule fois  
Ensuite : loop() sans fin

CHAÎNE D’INFORMATION DU VEILLEUR



La carte travaille en 3,3 V. Ne jamais appliquer 5 V sur une broche.

| Signal               | Broche | Rôle             |
|----------------------|--------|------------------|
| LDR (milieu du pont) | GPIO34 | Entrée du CAN    |
| LED + 220 Ω          | GPIO18 | Sortie numérique |
| Buzzer actif         | GPIO19 | Sortie numérique |

HIGH ≈ 3,3 V (niveau 1) · LOW ≈ 0 V (niveau 0)

2. LE CAN

Le convertisseur analogique-numérique transforme une tension en un nombre entier N.

- Pleine échelle Upe : tension maximale codée
- Résolution n : nombre de bits du code
- Nombre de codes : 2<sup>n</sup>
- Quantum q : plus petit échelon de tension

$$q = U_{pe} / 2^n$$
$$N \approx U / q$$
$$U \approx N \times U_{pe} / 2^n$$

Plus n est grand, plus q est petit : la mesure est plus fine. Deux tensions très proches peuvent donner le même nombre.

CAN DE L’ESP32 (PROJET)

| Grandeur                       | Valeur    |
|--------------------------------|-----------|
| Upe                            | 3,3 V     |
| n                              | 12 bits   |
| Nombre de codes 2 <sup>n</sup> | 4096      |
| N lu par analogRead            | 0 à 4095  |
| q = 3,3 / 4096                 | ≈ 0,81 mV |

Exemple : N = 2048

$$U \approx 2048 \times 3,3 / 4096 \approx 1,65 \text{ V}$$

Dans le programme du TP2 :

```
int N = analogRead(34);  
float U = (N / 4095.0) * 3.3;
```

GPIO34 est une entrée seule : on peut y lire le CAN, pas y écrire une sortie.

3. BASES DU C++ SUR LA CARTE

Le programme Arduino est écrit en C++. Chaque instruction se termine par un point-virgule. Un bloc d’instructions est entouré d’accolades {}. Un commentaire commence par // et n’est pas exécuté.

STRUCTURE ET TYPES

```
void setup() {
  // exécuté une fois
}
void loop() {
  // répété sans arrêt
}
```

| Type  | Rôle             | Exemple                 |
|-------|------------------|-------------------------|
| int   | entier           | int lum = 1800;         |
| float | nombre à virgule | float U = 1.65;         |
| bool  | vrai ou faux     | bool nuit = true;       |
| const | valeur fixe      | const int SEUIL = 1800; |

Le point décimal s’écrit avec un point : 3.3 et non 3,3.

INSTRUCTIONS UTILES

| Instruction                | Effet                        |
|----------------------------|------------------------------|
| pinMode(broche, OUTPUT)    | la broche devient une sortie |
| digitalWrite(broche, HIGH) | sortir 3,3 V                 |
| digitalWrite(broche, LOW)  | sortir 0 V                   |
| analogRead(broche)         | lire le CAN : 0 à 4095       |
| delay(150)                 | attendre 150 ms              |
| Serial.begin(115200)       | ouvrir le moniteur série     |
| Serial.println(lum)        | afficher une valeur          |

TESTS ET OPÉRATEURS

| Symbole | Sens                 | Exemple        |
|---------|----------------------|----------------|
| =       | affecter             | seuil = 1800   |
| ==      | égal à               | lum == SEUIL   |
| < >     | inférieur, supérieur | lum < SEUIL    |
| &&      | et                   | nuit && lavant |
| !       | non                  | !avant         |

```
if (lum < SEUIL) {
  digitalWrite(PIN_LED, HIGH);
} else {
  digitalWrite(PIN_LED, LOW);
}
```

Une variable bool mémorise l’état précédent : on détecte ainsi un changement (le bip du TP3), pas un niveau maintenu.

EXEMPLE — LED SELON LA LUMINOSITÉ

```
const int PIN_LDR = 34;
const int PIN_LED = 18;
const int SEUIL = 1800;

void setup() {
  pinMode(PIN_LED, OUTPUT);
}

void loop() {
  int lum = analogRead(PIN_LDR);
  if (lum < SEUIL) {
    digitalWrite(PIN_LED, HIGH);
  } else {
    digitalWrite(PIN_LED, LOW);
  }
}
```

SEUIL vient des mesures du TP2. Le sens < ou > dépend du câblage du pont.

À RETENIR

1. Le programme décide. Le matériel acquiert et agit.

2. analogRead renvoie un code, pas des volts. La tension se calcule.

3. q = Upe / 2°. Sur cet ESP32 : 3,3 V / 4096.

4. = range une valeur. == compare deux valeurs.

PIÈGES FRÉQUENTS

- Oublier pinMode avant digitalWrite
- Confondre le code CAN et la tension
- Utiliser GPIO34 comme une sortie
- Mettre 5 V sur une broche de l’ESP32
- Écrire = au lieu de == dans un if
- delay bloque tout le programme